KIT Department of Business and Economics
Institute for Operations Research (IOR)
Continuous Optimization
Prof. Dr. Oliver Stein

Seminar Paper

# Strategies to Avoid Poor Local Minima in Nonlinear Optimization

by

Nicolas Heidbrink
Matr. Nr.: 2538298
Industrial Engineering and Management (Bachelor)

Submission Date
09.06.2026

Supervisor: Stefan Schwarze, M. Sc.

# Declaration

I hereby certify that I have authored this thesis independently, that I have fully indicated all sources and aids used, and that I have marked all contents taken from the works of others. Furthermore, I have complied with the KIT Statutes for Safeguarding Good Research Practice in their currently valid version.

June 9, 2026

*Date* *Nicolas Heidbrink*

# Contents

# 1 Introduction

It is no secret that there are many applications for algorithms designed to find global minimal points of complex functions, ranging from machine learning to the natural sciences. While finding local minimal points is comparatively easy with methods that take advantage of gradients and second-order derivatives of objective functions, finding global minimal points is more complicated because local information obtained by evaluating a function at certain points is simply insufficient to make general statements about its entire geometry. Therefore, algorithms have been and are still being developed to find global minimal points.

In 1985, Levy and Montalvo published their paper "The Tunneling Algorithm for the Global Minimization of Functions," [3] in which they presented one such algorithm. Their algorithm takes an objective function, its gradient, and a bounding box as its input, and it returns a list of points believed to be global minimal points along with their objective function value. The authors of the paper focus on the numerical results of their algorithm at the expense of a deeper explanation of how the method works and what aspects may prevent it from working ideally; this makes an examination of how the algorithm may be improved more difficult. Therefore, we take examine the inner workings of the tunneling algorithm more closely, and then apply our insights in finding adaptations that aim to improve its effectivity. The algorithm in its original form and with adaptations were implemented in Python. The implementation is available in a GitHub repository [2].

# 2 Explanation of the Tunneling Algorithm

The tunneling algorithm as presented in Levy and Montalvo's paper [3] works by alternating between two phases: In the minimization phase, a minimization algorithm is run on the current iterate. The result is a local minimal point $x_i^\star$ with $f(x_i^\star) =: f^\star$.

In the tunneling phase, the goal is to find a new point in the feasible set of the problem with an objective function value equal to or less than that of the previous local minimal point. An auxiliary tunneling function is defined that takes positive values for points $x$ with $f(x) > f^\star$ and non-positive values for previously undiscovered points $x$ with $f(x) \leq f^\star$. The tunneling phase implements a descent method on the tunneling function and

terminates with the first point found with a non-positive tunneling function value.

## 2.1 The Tunneling Function

Levy and Montalvo define their tunneling function as [3, Eq. A.4]

$$T(x) = \frac{f(x) - f^\star}{\{\prod_{i=1}^{l}[(x - x_i^\star)^T(x - x_i^\star)]^{\eta_i}\}[(x - x_m)^T(x - x_m)]^{\lambda_0}}, \tag{1}$$

with the minimal previously found objective function value $f^\star$, discovered minimal points $x_i : f(x_i) = f^\star$, and another point $x_m$ designed to accelerate the algorithm by negating irrelevant local minimal points of the tunneling function.

The denominator is non-negative because it is the product of exponentiated squared norms. Therefore, the sign of the tunneling function is strictly dependent on the sign of the numerator: If $f(x) > f^\star$, then $T(x) > 0$; if $f(x) = f^\star$, then $T(x) = 0$; and if $f(x) < f^\star$, then $T(x) < 0$.

### 2.1.1 The First Term of the Denominator

The first term of the denominator, $\prod_{i=1}^{l}[(x-x_i^\star)^T(x-x_i^\star)]_i^{\eta_i}$, introduces poles at all previously discovered local minima $x_i$ that share the function value $f^\star$. That way, searches over $T(x)$ for new points with objective function values less than or equal to $f^\star$ will not return previously found points, and iterates are directed away from such points by descent methods.

The proof that an exponent $\eta_i$ which introduces a pole at $T(x_i^\star)$ can always be found is omitted from the paper, but the statement can be shown for twice continuously differentiable functions $f$ at isolated minimizers $x_i^\star$ as follows:

As previously stated, the sign of $T(x)$ depends only on its numerator, which is non-negative in a neighborhood of $x_i^\star$ for all $i$ because $f(x_i^\star) - f^\star = 0$ and $x_i^\star$ is a local minimal point of $f(x)$. Therefore, the points around any pole at a local minimal point will diverge to $+\infty$.

According to [5, Th. 2.1.30] , the Taylor expansion of the numerator around $x_i^\star$ is

$$f(x) = f(x_i^\star)+\nabla f(x_i^\star)^T(x-x_i^\star)+\frac{1}{2}(x-x_i^\star)^T D^2 f(x_i^\star)(x-x_i^\star)+\omega(x)\cdot\|x-x_i^\star\|^2,$$

with $\lim_{x \to x_i^\star} \omega(x) = 0$.

If $x_i^\star$ is a non-degenerate minimizer of $f(x)$ that satisfies the First Order Necessary Optimality Condition $\nabla f(x_i^\star) = 0$, e.g. a local minimal point in the interior of the feasible set, the Taylor expansion at that point can be rearranged to

$$\begin{aligned} f(x) - f^\star &= \frac{1}{2}(x - x_i^\star)^T D^2 f(x_i^\star)(x - x_i^\star) + \omega(x) \cdot \|x - x_i^\star\|^2 \\ &\geq (\lambda_{min}(D^2 f(x_i^\star)(x - x_i^\star)) + \omega(x)) \cdot \|x - x_i^\star\|^2. \end{aligned} \tag{2}$$

As $x \to x_i^\star$, $[(x - x_i^\star)^T (x - x_i^\star)]^{\eta_i}$ approaches 0 and the other terms in the denominator of $T(x)$ approach constant, finite values, meaning they can be ignored for an asymptotical examination of $T(x)$ at $x_i^\star$. The same applies to $(\lambda_{min}(D^2 f(x_i^\star)(x - x_i^\star)) + \omega(x))$ in the lower bound for the numerator in 2. Hence,

$$\begin{aligned} T(x) &\gtrsim \frac{\|x - x_i^\star\|^2}{[(x - x_i^\star)^T (x - x_i^\star)]^{\eta_i}} \\ &= \frac{\|x - x_i^\star\|^2}{\|x - x_i^\star\|^{2\eta_i}}. \end{aligned}$$

If $2\eta_i > 2$, then $\lim_{x \to x_i^\star} T(x) = \infty$ and there is a pole at $x_i^\star$.

If however $x_i^\star$ is a minimizer of $f(x)$ on the boundary of the feasible set, it no longer necessarily fulfills the First Order Necessary Optimality Condition. If the First Order Necessary Optimality Condition is not fulfilled, the Taylor expansion at that point can be rearranged to

$$\begin{aligned} f(x) - f^\star &= \nabla f(x_i^\star)^T (x - x_i^\star) + \omega(x) \cdot \|(x - x_i^\star)\| \\ &\geq -\|\nabla f(x_i^\star)\| \cdot \|(x - x_i^\star)\| + \omega(x) \cdot \|(x - x_i^\star)\| \\ &= (-\|\nabla f(x_i^\star)\| + \omega(x)) \cdot \|(x - x_i^\star)\| \end{aligned}$$

Similarly to the first case, terms approaching constant, finite values can be ignored and the convergence of $T(x)$ around $x_i^\star$ can be examined with the approximation

$$T(x) \gtrsim \frac{\|x - x_i^\star\|}{\|x - x_i^\star\|^{2\eta_i}}.$$

For $\|x - x_i^\star\| \to 0$, the limit of $T(x)$ is infinite if $2\eta_i > 1$.

In order to ensure that the first term of the denominator does not flatten the tunneling function by having all function values far from $x_i^\star$ be divided by a large denominator, the exponent $\eta_i$ is set to 0 for points $x$ not within a radius of about 1 from $x_i^\star$. The authors add a small ramp function to linearly transition the value of $\eta_i$ from whatever value was previously calculated to 0 over points $x$ with $\|x - x_i^\star\|_2 \in (1-\epsilon, 1+\epsilon)$ [3, Eq. A.5], where $\epsilon$ is a small parameter set to $10^{-5}$ in their implementation. They do not mention that the ramp function is not necessary to avoid discontinuities in $T(x)$; without it, the jump from one exponent to the other would occur at points where the base, and thus the entire exponentiation, is 1.

### 2.1.2 The Second Term of the Denominator

The second term of the denominator of (1), $[(x-x_m)^T(x-x_m)]^{\lambda_0}$, is intended to introduce poles in the vicinity of irrelevant local minima of the tunneling function. This helps prevent whatever algorithm is used to search for points $x$ with $T(x) \leq 0$ from getting stuck. In this section, the notation from [3] is adapted and indices are introduced for clarity.

After each descent step on $T_{i-1}(x)$ from $x^{(i-1)}$ to $x^{(i)}$, the location of the movable pole $x_m^{(i)}$ is set in [3, Eq. A.7] to

$$x_m^{(i)} := \begin{cases} x^{(i-1)} & \text{if } \|x^{(i)} - x^{(i-1)}\| < 1, \\ \xi_i x^{(i-1)} + (1-\xi_i)x^{(i)} & \text{if } \|x^{(i)} - x^{(i-1)}\| \geq 1, \end{cases} \tag{3}$$

with $\xi_i$ chosen so that $x_m^{(i)}$ is a point on the line from $x^{(i-1)}$ to $x^{(i)}$ with $\|x_m^{(i)} - x^{(i)}\| < 1$. ($T_i(x)$ denotes the tunneling function with the movable pole at $x_m^{(i)}$.) What—if any—influence this point has is determined by the exponent $\lambda_0^{(i)}$.

Firstly, the term [3, Eq. A.8]

$$u_i := (x^{(i)} - x^{(i-1)})^T(\tilde{x}^{(i+1)}(\lambda_0^{(i-1)}) - x^{(i)}), \tag{4}$$

is examined to determine whether the value of $\lambda_0^{(i-1)}$ from the previous iteration is sufficiently high to be kept as $\lambda_0^{(i)}$. $\tilde{x}^{(i+1)}(\lambda_0)$ is a function that represents what the next iterate $x^{(i+1)}$ would be if calculated with a given $\lambda_0$—in this case, if $\lambda_0^{(i)} = \lambda_0^{(i-1)}$. If $u_i < 0$, the potential step from $x^{(i)}$ to $\tilde{x}^{(i+1)}(\lambda_0^{(i-1)})$ points in the general direction back to $x^{(i-1)}$. This is taken to

mean that $\lambda_0^{(i-1)}$ was not high enough to be kept as $\lambda_0^{(i)}$, and it is iteratively increased by recalculating $\tilde{x}^{(i+1)}(\lambda_0)$ with higher values for $\lambda_0$ until $u_i \geq 0$ is satisfied.

If however $u_i \geq 0$ from the start, the two vectors are acute or orthogonal and point in the same general direction, meaning $\lambda_0^{(i-1)}$ was satisfactory. In this case, it is checked whether $\lambda_0^{(i)}$ can even be set to 0 to simplify the tunneling function. The potential displacement from $x^{(i)}$ to the following iterate if $\lambda_0^{(i)} = 0$, i.e. $\tilde{x}^{(i+1)}(0) - x^{(i)}$, is calculated and defines $\Delta x^{(i)}(0)$, and the displacement $\tilde{x}^{(i+1)}(\lambda_0^{(i-1)}) - x^{(i)}$ defines $\Delta x^{(i)}(\lambda_0^{(i-1)})$. With this information, $\lambda_0^{(i)}$ is set to 0 if $(\Delta x^{(i)}(0))^T \Delta x^{(i)}(\lambda_0^{(i-1)}) > 0$, or in other words, if the potential displacement vector without the movable pole forms an acute angle with the displacement vector assuming $\lambda_0$ stays constant from iteration $i-1$ to iteration $i$.

The paper does not provide the reason that $(x^{(i)} - x^{(i-1)})^T (x^{(i+1)} - x^{(i)}) < 0$ is taken as an indication that a local minimal point of the tunneling function was passed, meaning the value for $\lambda_0^{(i)}$ was not high enough. This can however be rationalized by the fact that an obtuse angle between two subsequent search directions $d_{i-1} := x^{(i)} - x^{(i-1)}$ and $d_i := x^{(i+1)} - x^{(i)}$ means that a smaller step size for $d_i$ would have produced a more optimal point $x^{(i+1)}$, assuming gradient descent is used to search for points with $T_i(x) \leq 0$. That statement can be shown as follows:

To examine the slope of $T_i(x^{(i)})$ along a search direction $d$ in iteration $i$, the one-dimensional restriction is defined as

$$\phi_d^{(i)}(t) := T_i(x^{(i)} + td).$$

Assuming gradient descent is used,

$$\phi^{(i)}_{-\nabla T_i(x^{(i)})}(t) = T_i(x^{(i)} - t\nabla T_i(x^{(i)})).$$

The slope of the one-dimensional restriction is

$$[\phi^{(i)}_{-\nabla T_i(x^{(i)})}]'(t) = (-\nabla T_i(x^{(i)}))^T \nabla T_i(x^{(i)} - t\nabla T_i(x^{(i)})).$$

The slope of $T_{i-1}(x)$ at $x^{(i-1)}$ along $-\nabla T_{i-1}(x^{(i-1)})$ can be determined by analogously deriving the one-dimensional restriction for iteration $i-1$ and substituting $t = 0$:

$$[\phi^{(i-1)}_{-\nabla T_{i-1}(x^{(i-1)})}]'(0) = -\nabla T_{i-1}(x^{(i-1)})^T \nabla T_{i-1}(x^{(i-1)}) < 0.$$

The slope of $T_{i-1}(x)$ at $x^{(i)} = x^{(i-1)} - t_{i-1}\nabla T_{i-1}(x^{(i-1)})$ along the same search direction can be expressed as

$$\begin{aligned}
[\phi^{(i-1)}_{-\nabla T_{i-1}(x^{(i-1)})}]'(t_{i-1}) &= (-\nabla T_{i-1}(x^{(i-1)}))^T \nabla T_{i-1}(x^{(i-1)} - t_{i-1}\nabla T_{i-1}(x^{(i-1)})) \\
&= (-\nabla T_{i-1}(x^{(i-1)}))^T \nabla T_{i-1}(x^{(i)}) \\
&\overset{(\star)}{\approx} -\left(-\frac{1}{t_{i-1}}(x^{(i)} - x^{(i-1)})\right)^T \left(-\frac{1}{t_i}(\tilde{x}^{(i+1)}(\lambda_0^{(i-1)}) - x^{(i)})\right) \\
&= -\frac{u_i}{t_{i-1}t_i}.
\end{aligned}$$

The approximation $(\star)$ in the above equation is not a perfect equality because the second factor of the product is equal to the gradient at $x^{(i)}$ of $T_i(x)$, not $T_{i-1}(x)$, meaning the location of the movable pole is not updated as it should be. This effect however plays no role when $\lambda_0^{(i-1)} = 0$, and because the evaluation of $u_i$ serves just as a heuristic, approximations do not pose a major problem.

If $u_i < 0$, then it seems therefore likely that $[\phi^{(i-1)}_{-\nabla T_{i-1}(x^{(i-1)})}]'(t_{i-1}) > 0$. As the one-dimensional restriction then has a negative slope at $x^{(i-1)}$ and a positive slope at $x^{(i)}$, it must have a local minimal point somewhere in between.

While movable poles can help prevent getting stuck in local minimal points with $T(x) > 0$, it can be observed that they do not guarantee it. For example, no pole would be created if each point in a sequence of iterates is the midpoint of the line segment between the previous iterate and a local minimal point.

Another drawback not mentioned by Levy and Montalvo is the risk that a local minimal point being repelled by a pole could be a valid termination point with $T(x) \leq 0$. This can occurs for example when the descent method used on the tunneling function overshoots a valid termination point and is prevented from directly taking a step back. In this case, the iterates have to move further away from that point before the pole is reset and the point can be reached again.

The proof that a $\lambda_0^{(i)}$ that leads to an acceptable $u$ can always be found is also omitted from the paper. Once again, we assume gradient descent is used, meaning

$$u_i = (x^{(i)} - x^{(i-1)})^T(-t_2\nabla T_i(x^{(i)})).$$

This means that $u_i \geq 0$ if and only if

$$(x^{(i)} - x^{(i-1)})^T \nabla T_i(x^{(i)}) \leq 0. \tag{5}$$

Summarizing the first term of its denominator as $P(x)$ and omitting the index $i$, the tunneling function is given as

$$T(x) = \frac{f(x) - f^\star}{P(x)[(x - x_m)^T(x - x_m)]^{\lambda_0}},$$

meaning the gradient after applying the quotient rule, the product rule, and the chain rule and subsequently substituting in $v := (x - x_m)^T(x - x_m)$ for legibility is

$$\begin{aligned}\nabla T(x) &= \frac{P(x)v^{\lambda_0}\nabla f(x) - (f(x) - f^\star)[v^{\lambda_0}\nabla P(x) + 2P(x)\lambda_0 v^{\lambda_0 - 1}(x - x_m)]}{P^2(x)v^{2\lambda_0}} \\ &= \frac{P(x)\nabla f(x) - (f(x) - f^\star)[\nabla P(x) + 2P(x)\lambda_0 v^{-1}(x - x_m)]}{P^2(x)v^{\lambda_0}}.\end{aligned}$$

For $\lambda_0 \to \infty$, the gradient approaches

$$-\frac{2\lambda_0(f(x) - f^\star)v^{-1}}{P(x)v^{\lambda_0}}(x - x_m), \tag{6}$$

which is $(x - x_m)$ multiplied with a negative constant because $P(x)$, $v$, and $(f(x) - f^\star)$ are all positive. $x_m^{(i)}$ was defined to be on the line segment connecting $x^{(i-1)}$ and $x^{(i)}$ in (3), so $-(x^{(i)} - x_m^{(i)})$ is antiparallel to $(x^{(i)} - x^{(i-1)})$. The condition (5) is thus fulfilled for sufficiently large $\lambda_0^{(i)}$.

## 2.2 Applying the Algorithm

To find points $x$ with $T(x) \leq 0$ starting at the point returned in the previous minimization phase, the tunneling function is constructed with a pole at the starting point [3, pp. 26–27]. A small random perturbation is added because descent methods applied in the tunneling phase can generally not start from discontinuities.

While any descent method can be chosen, the authors suggest a modified Newton's method [3, p. 28], where displacements are determined as [3, §2.3.4]

$$\Delta x = -\alpha \frac{T(x)}{\|\nabla T(x)\|^2} \nabla T(x), \tag{7}$$

with a step size parameter $\alpha$ determined with bisection starting at $\alpha = 1$.

We observe that each iterate is guaranteed to decrease $T(x)$ because the method is a special form of gradient descent and step sizes can be as small as necessary to have a descent step; however, this does not necessarily mean that a sequence of iterates will lead to points closer and closer to being acceptable termination points for the tunneling phase. The movable pole can cause the method to diverge by ensuring two subsequent iterates lead to an improvement on the tunneling function given a certain $x_m$ even if the second iterate would be less preferable without the movable pole. For instance, points $x^{(1)}, x^{(2)}, x^{(3)}$ with $T_1(x^{(1)}) > T_1(x^{(2)})$ and $T_2(x^{(2)}) > T_2(x^{(3)})$ could be consecutive iterates even if $f(x^{(1)}) < f(x^{(2)}) < f(x^{(3)})$. While this can help avoid getting stuck in local minimal points of the tunneling function, it means there is no guarantee the tunneling phase will terminate with an acceptable point.

Accounting for this, the authors recommend starting the tunneling phase from up to $2n$ random perturbations, where $n$ is the amount of dimensions in the domain of the objective function. If all should fail to find an acceptable point, the authors suggest starting a tunneling phase from another $2n$ points randomly selected from the entire domain. Only if all of these fail to find a new iterate does the entire tunneling algorithm terminate.

# 3 Limitations of the Tunneling Algorithm

While Levy and Montalvo report impressive results [3, §5], they do not elaborate on issues with the tunneling algorithm that can in certain cases lead it to fail to find global minima even for simple functions.

For one, only up to $2n$ different random perturbations being applied to start off the modified Newton's method means certain search directions can be neglected if the perturbations happen to point in similar directions. This can be fixed by increasing the number of random perturbations, or by removing the randomness and ensuring the perturbations point in different directions. Increasing the number of random perturbations means the algorithm will take longer to terminate because it becomes less likely none of the perturbations will result in a new optimal point.

Another problem arises when applying the modified Newton's method to tunneling functions with a movable pole. To illustrate this issue, we examine the case of a one-dimensional function $f(x)$ being optimized with

the tunneling algorithm. If the tunneling phase has reached an area not affected by poles around previously found minimal points and a movable pole is necessary to maintain the search direction, the tunneling function will be given as

$$T(x) = \frac{f(x) - f^\star}{(x - x_m)^{2\lambda_0}}. \tag{8}$$

Its derivative in this one-dimensional example is

$$T'(x) = \frac{f'(x)(x - x_m)^{2\lambda_0} - (f(x) - f^\star)2\lambda_0(x - x_m)^{2\lambda_0 - 1}}{(x - x_m)^{4\lambda_0}} \tag{9}$$

Thus, the displacement according to (7) is

$$\Delta x = -\alpha \frac{f(x) - f^\star}{f'(x)(x - x_m) - 2\lambda_0(f(x) - f^\star)}(x - x_m). \tag{10}$$

If $f(x) - f^\star >> f'(x)(x - x_m)$, which is the case if the current iterate is far from being optimal, if the current iterate is close to the movable pole, or if the slope of the objective function is small, then the term $f'(x)(x - x_m)$ does not have much of an impact on the fraction in (10), which can then be approximated as

$$\Delta x \approx \frac{1}{2\lambda_0}\alpha(x - x_m). \tag{11}$$

If the step size $\alpha$ is set to 1 and potentially even decreased with bisection, as suggested in [3, §2.3.4], the distance from a first iterate to a second iterate will be at most about half the distance between the movable pole and the first iterate, since $\lambda_0 \geq 1$. As the next movable pole is then set to the aforementioned first iterate, the distance between the second and third iterate will be at most about half the distance between the first and the second. This pattern makes the steps quickly become arbitrarily small and stalls the algorithm. The problem also seems to arise for functions $f(x)$ with multidimensional domains, perhaps because they behave similarly to one-dimensional restrictions along their gradients when applying gradient descent algorithms. The modified Newton's method stalling because displacements were approximately halved was observed in an experiment with a three-dimensional function [2].

To solve the problem of vanishing step sizes, several possible solutions come to mind. Increasing the pre-bisection value of $\alpha$ to 2 would stop the displacement between iterates approximated in (11) from decreasing with every iteration. However, decreases in the displacement of one iterate like those

due to bisection would still affect subsequent iterates. Thus, one bisection would cause a lasting slowdown.

An alternative approach that experimentally shows preferable results is fixing the movable pole so that $\|x - x_m\|_2$ is constant, while keeping it on the line that connects $x$ to the previous iterate. This disrupts the validity of approximating (10) with (11). Setting $\|x - x_m\|_2$ to be close to but less than 1 ensures that the approximation is invalid and minimizes nonconformity to Levy and Montalvo's implementation, which asserts that the movable pole must be in the interior of a unit ball around $x$ [3, p. 27].

The non-random perturbations and fixed distances from $x_m$ were implemented and, together with the original version of the tunneling function, tested against a variety of objective functions [2]. The objective functions the algorithms were tested against are described in Section A.1. The results of the tests on the unaltered implementation of the algorithm and the implementation with the adaptations can be seen in Section A.3 and Section A.4, respectively. The adapted version performed as well as or significantly better than the unaltered version for all test functions.

# 4 Comparison to Iterated Local Search

Iterated local search methods as presented in [4] are optimization algorithms that loop between applying a local search method to an objective function and then find another point from which to start another iteration of local search. The method with which starting points for local search are determined is left open; Iterated Local Search itself is a metaheuristic.

Levy and Montalvo's tunneling algorithm can be identified as such an Iterated Local Search method, where the method to choose starting points for local search in the minimization phase is the tunneling phase. It is therefore logical to consider applying techniques for improving iterated local search methods presented in [4] to the tunneling algorithm to see if there is still more room for improvement.

## 4.1 ILS Features Already Implemented in the Tunneling Algorithm

Lourenço et al. mention random restart as a simple possibility to find more optimal points [4, §5.2.2]. This is indirectly included as a feature of the tunneling algorithm: When the perturbations of the previous minimal point fail to locate a new minimal point, a global search phase is initiated, in which a descent method is applied on random points of the domain of the tunneling function. What separates this implementation by Levy and Montalvo from the idea described by Lourenço et al. is the fact that the descent method starting from the random points is applied on the tunneling function instead of the original objective function. This means that information about previous minimal points is not ignored and the search is again prevented from terminating with suboptimal local minimal points.

Another central point raised in [4] is that incorporating memory in the search for new points to start minimization phases from improves performance. Especially for functions with lots of global minimal points sharing an objective function value, the location of previously found minimal points being saved and incorporated as poles is such a form of memory. This also contributes to another desirable attribute of iterated local search methods brought up by Lourenço et al.: the fact that they should not be able to return to previously visited optima.

## 4.2 Simulated Annealing

Another technique the authors mention is simulated annealing: Instead of searching only for points as good as or better than the previously found optimal point to start a minimization phase from, inferior points are used as the starting point of minimization phase with a probability that depends positively on how close they are to being optimal and a so-called temperature $c_i$ that decreases over time [4, §5.3.3]. The formula typically used for an already discovered minimal point $x_i$ and a new point $x_j$ is given in [1, Def. 5]:

$$P\{\text{accept } x_j\} = \begin{cases} 1 & \text{if } f(x_j) < f(x_i) \\ \exp\left(\frac{f(x_i)-f(x_j)}{c}\right) & \text{else.} \end{cases} \tag{12}$$

In the tunneling algorithm, implementing simulated annealing means adding

the possibility of terminating tunneling phases with points that do not satisfy $T(x) \leq 0$. This effectively means switching from a descent method on $T(x)$ to a descent method on $f(x)$ before it is guaranteed that the descent method on $f(x)$ will find a new optimal point. This allows more efficient descent methods implemented on the simpler $f(x)$ to take over quicker. By accepting more points during the tunneling phase, the risk of the entire algorithm terminating prematurely is also decreased because there is less of a risk of a tunneling phase failing to find a new acceptable point. The effect is therefore similar to increasing the parameters that determines how many perturbations and global searches are attempted from a given previously found minimizer.

Another advantage of adding some kind of alternative termination criteria, as in simulated annealing, is that it removes the necessity of a movable pole. When the algorithm runs into a local minimal point of the tunneling function, the movable pole is required to prevent the algorithm from stalling. Introducing the possibility of tunneling phases terminating prematurely means the stalling can be interrupted by a new minimization phase. It must however be noted that the risk of the algorithm stalling around a local minimal point for many iterations before being interrupted then rises as the temperature decreases over time. The risk of uninterrupted stalling also increases for local minimal points close to being optimal.

Experiments with a version of the tunneling algorithm without a movable pole and with simulated annealing were run [2] on the basket of tests described in Section A.1. The version of simulated annealing implemented was inspired by geometric cooling, as described in [1, §1.4.2]. In geometric cooling, the initial temperature should be set so that most transitions would be accepted at the start of the algorithm. This was implemented by taking a uniformly distributed random sample of 100 points from the feasible set of the optimization problem, calculating their objective function values, and sorting them by objective function value. The lowest objective function value $f(x_1)$ and the tenth lowest objective function value $f(x_{10})$ are taken as rough estimates for the objective function value of an optimal point and the objective function value of the next iterate in a tunneling phase. Pinning the initial probability of such a next iterate being accepted to 90%, then substituting the corresponding values into 12, and finally solving the resulting equation

$$P\{\text{accept } x_{10}\} = 90\% = \exp \frac{f(x_1) - f(x_{10})}{c_0}$$

leads to an initial temperature of

$$c_0 = \frac{f(x_1) - f(x_{10})}{\ln 0.9}.$$

The formula implemented for the decay of the temperature over time was taken directly from the description of geometric cooling by Delayahe et al. with a parameter of $\alpha = 0.95$, meaning

$$c_i = 0.95 \times c_{i-1}.$$

The results of the tests are displayed in Section A.5. Incorporating simulated annealing resulted in a notable increase in time needed per optimization problem. It improved performance compared to the strict implementation of Levy and Montalvo's algorithm, but was still outperformed by the adapted version described in Section 3. The large amount of time per trial and the smaller amount of global minimal points found on average, especially in functions where many exist (e.g. $f_2$, $f_3$), indicate that the algorithm did not explore enough of the domain; perhaps points still in previously explored areas were too often accepted as termination points for tunneling phases by the simulated annealing condition. Increasing the total amount of points explored did still lead to some improvement. It seems that the algorithm became somewhat more similar to a Multiple Random Start [3, §4.1.2] method once simulated annealing was added: Even when the initial points of minimization phases were not chosen as methodically as before, simply increasing the amount of minimization phases led to improvement. (The cited Multiple Random Start method randomly selects a large number of initial points on which local minimization algorithms are applied. The resulting local minimal points with the smallest objective function value are assumed to be optimal.)

# 5 Conclusion

The tests indicate that, while not perfect in its original form, the movable pole is an effective way to ensure the tunneling algorithm does not remain in areas already explored. It appears that the randomness of the perturbations applied to the previously found minimal point at the start of each tunneling phase was enough to move into new areas of the feasible set.

An question still open is why the implementation by the inventors of the method performed better. It is not plausible that the only reason was differences in the testing methods applied, such as the inventors choosing starting

points of the algorithm in advance instead of generating them randomly, as was done in [2]; by the time one global minimal point is found, the memory of previous iterates is erased. Perhaps lower floating-point precision prevented effects like those described in Section 3 from taking effect; this however seems unlikely because some of the algorithm parameters were set as low as $10^{-9}$, e.g. in [3, §4.1.1].

Another question that arises is whether the algorithm can be applied to optimization problems with more complex constraints. Incorporating a penalty function seems like a simple solution, but it remains unclear to what extent it would negatively impact the stability of the algorithm, especially in combination with the movable pole. A sequence of iterates moving towards the boundary of a feasible set could be repelled away from the interior of the feasible set by the movable pole and repelled away from infeasible points by a penalty function at the same time, causing very instable results

# A Annex: Algorithm Tests

## A.1 Functions Used to Test the Algorithms

Table 1 contains the functions used to test the algorithms. Functions $f_1$, $f_2$, and $f_3$ are implementations of the Shubert function on 2, 3, and 6 dimensions, respectively. Function $f_4$ is the Six Hump Camel function presented in [3]. Function $f_5$ is the an altered version of $f_1$ that has only one global minimal point [3, §4.4.2]. Functions $f_6$ to $f_8$ are also taken from [3, Eq. 15] with 2, 3, and 4 dimensions, and $f_9$, $f_{10}$, and $f_{11}$ are implementations of [3, Eq. 16] with 5, 8, and 10 dimensions. The final six functions are implementations of [3, Eq. 17] over two through seven dimensions.

| Function | Runs | Bounds | Minimal Value | Amt. of Minima |
|---|---|---|---|---|
| $f_1$ | 40 | $[-10, 10]^2$ | -186.731 | 18 |
| $f_2$ | 10 | $[-10, 10]^3$ | -2709.094 | 81 |
| $f_3$ | 10 | $[-10, 10]^6$ | -8272701.378 | 4374 |
| $f_4$ | 30 | $[-3, 3] \times [-2, 2]$ | -1.032 | 2 |
| $f_5$ | 30 | $[-10, 10]^2$ | -186.731 | 1 |
| $f_6$ | 30 | $[-10, 10]^2$ | 0.0 | 1 |
| $f_7$ | 20 | $[-10, 10]^3$ | 0.0 | 1 |
| $f_8$ | 10 | $[-10, 10]^4$ | 0.0 | 1 |
| $f_9$ | 10 | $[-10, 10]^5$ | 0.0 | 1 |
| $f_{10}$ | 10 | $[-10, 10]^8$ | 0.0 | 1 |
| $f_{11}$ | 10 | $[-10, 10]^{10}$ | 0.0 | 1 |
| $f_{12}$ | 10 | $[-10, 10]^2$ | 0.0 | 1 |
| $f_{13}$ | 10 | $[-10, 10]^3$ | 0.0 | 1 |
| $f_{14}$ | 10 | $[-10, 10]^4$ | 0.0 | 1 |
| $f_{15}$ | 10 | $[-10, 10]^5$ | 0.0 | 1 |
| $f_{16}$ | 10 | $[-10, 10]^6$ | 0.0 | 1 |
| $f_{17}$ | 10 | $[-10, 10]^7$ | 0.0 | 1 |

Table 1: Test functions

## A.2 Results Reported by Levy and Montalvo

In [3, Table 1], the results below are reported. The authors did not test their implementation on $f_2$ or $f_3$. The average time per run is omitted due to incomparability, and the rate of failing to find a single global minimal point is omitted because it was not reported.

| Test | Avg. Minima Found |
|---|---|
| $f_1$ | 17 / 18 |
| $f_2$ | n/a |
| $f_3$ | n/a |
| $f_4$ | 2 / 2 |
| $f_5$ | 1 / 1 |
| $f_6$ | 1 / 1 |
| $f_7$ | 1 / 1 |
| $f_8$ | 1 / 1 |
| $f_9$ | 1 / 1 |
| $f_{10}$ | 1 / 1 |
| $f_{11}$ | 1 / 1 |
| $f_{12}$ | 0.5 / 1 |
| $f_{13}$ | 0.75 / 1 |
| $f_{14}$ | 0.75 / 1 |
| $f_{15}$ | 0.75 / 1 |
| $f_{16}$ | 1 / 1 |
| $f_{17}$ | 0.75 / 1 |

Table 2: Performance metrics reported by Levy and Montalvo.

## A.3 Unaltered Tunneling Algorithm

Staying true to the description in [3], the results do not match the results reported by Levy and Montalvo; the non-zero failure rates in the functions starting with $f_7$ and the low average number of minima found for $f_1$, where a reported 17 out of 18 were found in the authors' implementation, show significantly worse performance.

Comparing the time per trial is not feasible due to the different hardware used and the fact that the implementation in [2] was not systematically optimized for performance.

| Test | Avg. Minima Found | Failure Rate | Time/Trial |
|---|---|---|---|
| $f_1$ | 1.52 / 18 | 17.50% | 0.467 s |
| $f_2$ | 0.40 / 81 | 60.00% | 0.458 s |
| $f_3$ | 0.00 / 4374 | 100.00% | 0.711 s |
| $f_4$ | 1.47 / 2 | 0.00% | 0.082 s |
| $f_5$ | 0.13 / 1 | 86.67% | 0.341 s |
| $f_6$ | 1.00 / 1 | 0.00% | 0.132 s |
| $f_7$ | 0.85 / 1 | 15.00% | 0.233 s |
| $f_8$ | 0.20 / 1 | 80.00% | 0.436 s |
| $f_9$ | 0.50 / 1 | 50.00% | 0.497 s |
| $f_{10}$ | 0.10 / 1 | 90.00% | 0.516 s |
| $f_{11}$ | 0.20 / 1 | 80.00% | 0.870 s |
| $f_{12}$ | 0.40 / 1 | 60.00% | 0.113 s |
| $f_{13}$ | 0.00 / 1 | 100.00% | 0.255 s |
| $f_{14}$ | 0.00 / 1 | 100.00% | 0.244 s |
| $f_{15}$ | 0.30 / 1 | 70.00% | 0.350 s |
| $f_{16}$ | 0.20 / 1 | 80.00% | 0.391 s |
| $f_{17}$ | 0.00 / 1 | 100.00% | 0.442 s |

Table 3: Performance metrics for unaltered tunneling algorithm.

## A.4 Adapted Tunneling Algorithm

With the changes mentioned in Section 3, the algorithm performs much better, as seen in the table below. The increase in time needed per trial can at least partially be attributed to the fact that the algorithm no longer terminates prematurely. The algorithm still does not quite match the performance reported by Levy and Montalvo when applied to $f_1$, although it surpasses their performance when applied to the functions starting with $f_{12}$.

| Test | Avg. Minima Found | Failure Rate | Time/Trial |
|---|---|---|---|
| $f_1$ | 13.68 / 18 | 0.00% | 1.834 s |
| $f_2$ | 13.30 / 81 | 0.00% | 1.031 s |
| $f_3$ | 3.80 / 4374 | 60.00% | 1.303 s |
| $f_4$ | 2.00 / 2 | 0.00% | 0.206 s |
| $f_5$ | 0.93 / 1 | 6.67% | 0.506 s |
| $f_6$ | 1.00 / 1 | 0.00% | 0.158 s |
| $f_7$ | 1.00 / 1 | 0.00% | 0.243 s |
| $f_8$ | 1.00 / 1 | 0.00% | 0.328 s |
| $f_9$ | 1.00 / 1 | 0.00% | 0.473 s |
| $f_{10}$ | 1.00 / 1 | 0.00% | 1.052 s |
| $f_{11}$ | 1.00 / 1 | 0.00% | 0.933 s |
| $f_{12}$ | 1.00 / 1 | 0.00% | 0.194 s |
| $f_{13}$ | 1.00 / 1 | 0.00% | 0.327 s |
| $f_{14}$ | 1.00 / 1 | 0.00% | 0.416 s |
| $f_{15}$ | 1.00 / 1 | 0.00% | 0.458 s |
| $f_{16}$ | 1.00 / 1 | 0.00% | 0.534 s |
| $f_{17}$ | 1.00 / 1 | 0.00% | 0.592 s |

Table 4: Performance metrics for adapted tunneling algorithm.

## A.5 Tunneling Algorithm with Simulated Annealing

With simulated annealing implemented on the tunneling algorithm instead of the movable pole, the algorithm performs better than in its original version; however, it also tends to take longer to terminate. The implementation does not perform as well as the adapted version of the algorithm described in Section 3.

| Test | Avg. Minima Found | Failure Rate | Time/Trial |
|---|---|---|---|
| $f_1$ | 10.85 / 18 | 0.00% | 0.877 s |
| $f_2$ | 5.10 / 81 | 0.00% | 1.105 s |
| $f_3$ | 0.10 / 4374 | 90.00% | 1.267 s |
| $f_4$ | 1.50 / 2 | 0.00% | 0.118 s |
| $f_5$ | 0.67 / 1 | 33.33% | 0.388 s |
| $f_6$ | 1.00 / 1 | 0.00% | 0.421 s |
| $f_7$ | 0.95 / 1 | 5.00% | 1.698 s |
| $f_8$ | 0.90 / 1 | 10.00% | 2.303 s |
| $f_9$ | 1.00 / 1 | 0.00% | 2.141 s |
| $f_{10}$ | 0.80 / 1 | 20.00% | 3.584 s |
| $f_{11}$ | 0.80 / 1 | 20.00% | 6.316 s |
| $f_{12}$ | 1.00 / 1 | 0.00% | 0.512 s |
| $f_{13}$ | 1.00 / 1 | 0.00% | 1.147 s |
| $f_{14}$ | 1.00 / 1 | 0.00% | 2.114 s |
| $f_{15}$ | 1.00 / 1 | 0.00% | 3.144 s |
| $f_{16}$ | 1.00 / 1 | 0.00% | 3.048 s |
| $f_{17}$ | 1.00 / 1 | 0.00% | 4.653 s |

Table 5: Performance metrics for tunneling algorithm with simulated annealing.

# References

[1] Daniel Delahaye, Supatcha Chaimatanan, and Marcel Mongeau, "Simulated Annealing: From Basics to Applications", en, *Handbook of Metaheuristics*, ed. by Michel Gendreau and Jean-Yves Potvin, Cham: Springer International Publishing, 2019, pp. 1–35, ISBN: 978-3-319-91086-4, DOI: 10.1007/978-3-319-91086-4_1, https://doi.org/10.1007/978-3-319-91086-4_1 (visited on 06/08/2026).

[2] Nicolas Heidbrink, *nicolasheidbrink/optimization-seminar*, original-date: 2026-05-14T16:11:09Z, May 2026, https://github.com/nicolasheidbrink/optimization-seminar (visited on 06/04/2026).

[3] A. V. Levy and A. Montalvo, "The Tunneling Algorithm for the Global Minimization of Functions", *SIAM Journal on Scientific and Statistical Computing* 6.1 (Jan. 1985), pp. 15–29, ISSN: 0196-5204, DOI: 10.1137/0906002, https://epubs.siam.org/doi/10.1137/0906002 (visited on 05/01/2026).

[4] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle, "Iterated Local Search", en, *Handbook of Metaheuristics*, ed. by Fred Glover and Gary A. Kochenberger, Boston, MA: Springer US, 2003, pp. 320–353, ISBN: 978-0-306-48056-0, DOI: 10.1007/0-306-48056-5_11, https://doi.org/10.1007/0-306-48056-5_11 (visited on 05/18/2026).

[5] Oliver Stein, *Basic Concepts of Nonlinear Optimization*, eng, 1st ed. 2024, Mathematics Study Resources 8, Berlin, Heidelberg: Springer Berlin Heidelberg, 2024, ISBN: 978-3-662-69741-2, DOI: 10.1007/978-3-662-69741-2.